

Scripting in bash

Annie Meyer

Feb 10, 2026

Today's goal:

- Take a specific script we wrote earlier and make it usable in different contexts
- consider how different types of scripts are used, why you might want to use one vs the other

3 main types of bash scripts

- Executable- “here is exactly how I did something”
- Template- “here’s how to do the thing with different inputs”
- Program – “heres how to do the thing generally”
 - Can be easily integrated into other scripts or pipeliens

Things to keep in mind

- Reproducibility
- Reusability
- Encapsulation and abstraction (user friendliness, utility)
- Documentation/ readability

Executable programs

- Reproducibility! Replicate results!
- Run the script, get the same output each time!
- Example: Some journals require you to include the code you used to generate certain figures, results, etc. so others can replicate your results

Our starting program: yeast_feature_counts.sh

```
# yeast_feature_counts.sh
# Purpose: count feature types in yeast genome annotation GFF file
# Output: a CSV table sorted by freq of feature

wget -q
https://ftp.ncbi.nlm.nih.gov/genomes/refseq/fungi/Saccharomyces_cerevisiae/reference/GCF_00014604
5.2_R64/GCF_000146045.2_R64_genomic.gff.gz

gunzip GCF_000146045.2_R64_genomic.gff.gz

awk -F"\t" -v OFS="," \
    'NF == 9 {cts[$3] += 1}
    END {for (ftr in cts)
        print ftr, cts[ftr] } ' \
    GCF_000146045.2_R64_genomic.gff |
sort -t, -k 2 -nr
```

TERMINAL

\$ sh yeast_feature_counts.sh

Template programs

- Make our script conveniently reusable!
- Re-write the script to include variables for specifying the input, output file names, parameters, etc.
- Easily plug in new variables and it will run the same analysis
- Example: I usually align my RNA-seq reads using specific parameters, so I have a template I fill out when I get a new sample

TERMINAL

```
$ cp yeast_feature_count.sh template_feature_count.sh
```

Turning yeast_feature_counts.sh into feature_count_template.sh

```
# template_feature_counts.sh

# Purpose: count feature types in a genome annotation GFF file

# Output: a CSV table sorted by freq of feature

##Set VARS

OUT_DIR="output" # Directory for output, will be created if necessary

GFF_URL="" # RefSeq URL Location

#check if output directory exists, if not create it

if [ ! -e ${OUT_DIR} ]; then

    mkdir -p ${OUT_DIR}

fi

# intermediate files written to OUT_DIR

gff_gz=${OUT_DIR}/gff.gz

gff_file=${OUT_DIR}/features.gff
```



```
# download the data if needed

if [ ! -f ${gff_gz} ]; then

    wget ${GFF_URL} -O ${gff_gz}

fi

# and unzip the data if needed

if [ ! -f ${gff_file} ]; then

    gunzip ${gff_gz} -c > ${gff_file}

fi

awk -F"\t" -v OFS="," \

    'NF == 9 {cts[$3] += 1}

    END {for (ftr in cts)

        print ftr, cts[ftr] }' \

    ${gff_file} |

sort -t, -k 2 -nr
```

Applying our template to count features in E. coli

TERMINAL

```
$ cp template_feature_counts.sh. ecoli_feature_counts.sh
```

Modify template

```
# ecoli_feature_counts.sh

# Purpose: count feature types in an ecoli genome annotation GFF file

# Output: a CSV table sorted by freq of feature

##Set VARS

OUT_DIR="Escherichia_coli" # Directory for output, will be created if necessary

GFF_URL="https://ftp.ncbi.nlm.nih.gov/genomes/refseq/bacteria/Escherichia_coli/reference/GCF_000005845.2_ASM584v2/GCF_000005845.2_ASM584v2_genomic.gff.gz" # RefSeq URL
Location

#check if output directory exists, if not create it

if [ ! -e ${OUT_DIR} ]; then
    mkdir -p ${OUT_DIR}
fi

# intermediate files written to OUT_DIR

gff_gz=${OUT_DIR}/gff.gz
gff_file=${OUT_DIR}/features.gff
```



```
# download the data if needed
if [ ! -f ${gff_gz} ]; then
    wget ${GFF_URL} -O ${gff_gz}
fi

# and unzip the data if needed
if [ ! -f ${gff_file} ]; then
    gunzip ${gff_gz} -c > ${gff_file}
fi

awk -F"\t" -v OFS="," \
    'NF == 9 {cts[$3] += 1}
    END {for (ftr in cts)
        print ftr, cts[ftr] }' \
    ${gff_file} |
sort -t, -k 2 -nr
```

Run template to get counts for e coli!

TERMINAL

```
$ sh ecoli_feature_count.sh
```

Creating an easy-to-use program

- Simplified, easy to use version of the program that can be integrated into other pipelines
- Encapsulation and abstraction
 - Neatly bundled up and ready to go
 - Nitty gritty details of analysis aren't as important
 - All a user needs to know is what data to run the program on and what the output is
- Example: I look at a lot of phylogenetic trees and have a terrible time remembering node names. I can call a program that will tell me the name of the last common ancestor of two species

Specifying input from the command line

TERMINAL

```
$ cp template_feature_count.sh feature_count.sh
```

```
#!/usr/bin/env bash
```

```
#feature_count.sh
```

```
# Purpose: based on user input, download a GFF file from refseq and  
count feature types
```

```
# Output: a CSV table sorted by freq of feature
```

```
nargs=$#
```

```
if [ $nargs -ne 2 ]; then
```

```
echo "usage: $0 OUTDIR URL"
```

```
exit 1
```

```
fi
```

```
OUT_DIR=$1 # Directory for output, will be created if necessary
```

```
GFF_URL=$2 # RefSeq URL Location
```

```
# Create output directory if it doesn't exist
```

```
if [ ! -e ${OUT_DIR} ]; then
```

```
    mkdir -p ${OUT_DIR}
```

```
fi
```

```
# intermediate files written to OUT_DIR
```

```
gff_gz=${OUT_DIR}/gff.gz
```

```
gff_file=${OUT_DIR}/features.gff
```

```
# download the data if needed
```

```
if [ ! -f ${gff_gz} ]; then
```

```
    wget ${GFF_URL} -O ${gff_gz}
```

```
fi
```

```
# and unzip the data if needed
```

```
if [ ! -f ${gff_file} ]; then
```

```
    gunzip ${gff_gz} -c > ${gff_file}
```

```
fi
```

```
# run the analysis sending the results to stdout
```

```
awk -F"\t" -v OFS="," \
```

```
    'NF == 9 {cts[$3] += 1}
```

```
    END {for (ftr in cts)
```

```
        print ftr, cts[ftr] }' \
```

```
    ${gff_file} |
```

```
sort -t, -k 2 -nr
```

TERMINAL

```
$ sh template_feature_count.sh celegens \  
https://ftp.ncbi.nlm.nih.gov/genomes/refseq/invertebrate/Caenorhabditis_elegans/reference/GCF_00000298  
5.6_WBcel235/GCF_000002985.6_WBcel235_genomic.gff.gz
```

Keeping a record of our data

TERMINAL

```
$ touch genomelist.csv
```

```
Saccharomyces_cerevisiae,https://ftp.ncbi.nlm.nih.gov/genomes/refseq/fungi/Saccharomyces_cerevisiae/reference/GCF_000146045.2_R64/GCF_000146045.2_R64_genomic.gff.gz
```

```
Escherichia_coli,https://ftp.ncbi.nlm.nih.gov/genomes/refseq/bacteria/Escherichia_coli/reference/GCF_000005845.2_ASM584v2/GCF_000005845.2_ASM584v2_genomic.gff.gz
```

```
Caenorhabditis_elegans,https://ftp.ncbi.nlm.nih.gov/genomes/refseq/invertebrate/Caenorhabditis_elegans/reference/GCF_000002985.6_WBcel235/GCF_000002985.6_WBcel235_genomic.gff.gz
```

Format is genus_species,GFFURL

Creating a script to run feature_counts.sh on the different species in our .csv

TERMINAL

```
$ touch multispecies_feaure_count.sh
```

```
#!/bin/bash
# multispecies_count.sh
#purpose: run feature_count.sh on a csv file containing genomes of intrest
#output: for each species, a folder containing the downloaded .gff and ftr_counts.csv

while IFS="," read OUTDIR URL
do
    # we need this check because our redirection call below
    # will try and reference OUTDIR before the directories are referenced i
    # in feature_count.sh
    if [ ! -e ${OUTDIR} ]
    then
        mkdir -p ${OUTDIR}
    fi
    bash feature_count.sh ${OUTDIR} ${URL} > ${OUTDIR}/ftr_counts.csv
done < genomelist.csv
```

Running multi_species_count.sh

TERMINAL

```
$ sh multispecies_count.sh
```

Want to run it on a new species? Just add it to genomelist.csv!

Anyone can run the program as long as they have:

- A csv file of species names and gff urls
- feature_counts.sh script
- multispecies_count.sh script

